Custom GPU Architecture Specification v0.2

Siam Al Shafin · Version 0.2 · October 2026

1. Overview

This GPU renders textured 3D scenes at 1280×720 and 30 frames per second, using a full classic graphics pipeline, a custom instruction set, and fixed-point math throughout. It is designed from scratch by one engineer, targets the ASAP7 7 nm predictive PDK for physical design, and is proven first in simulation, then on an FPGA board.

Design goals

- Complete classic pipeline: vertex shading, clipping and culling, rasterization, perspective-correct textures, depth and stencil, blending, display output.
- Smallest hardware that meets 720p at 30 fps: 2 compute units with 4 lanes each.
- Fixed-point arithmetic only (no floating-point units), with a bit-exact Python reference model.
- Unified shader cores: the same compute units run vertex and pixel programs.
- A reserved slot for a future upscaler from 720p to 1080p, first a simple filter, later an AI model.

In scope for v0.1: everything in sections 2 to 9.

Out of scope for v0.1: geometry and tessellation shaders, ray tracing, anti-aliasing (MSAA), anisotropic filtering, floating-point units, standard graphics APIs (OpenGL, DirectX, Vulkan).

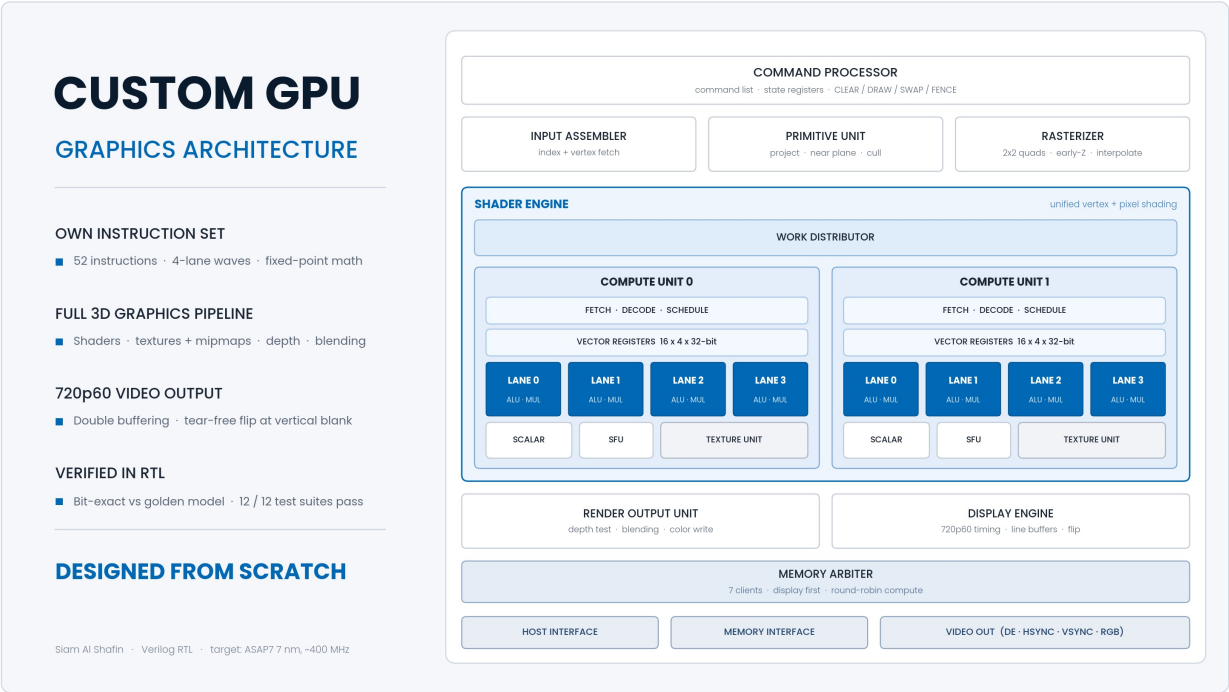
2. Targets and key numbers

The GPU renders 1280×720 at 30 fps with 8 shader lanes at about 400 MHz, giving roughly 6× headroom over the estimated shading load.

Item	Target	Note
Render resolution	1280 × 720	Parameter; tests run at 64 × 36
Frame rate	30 fps rendered	Each frame shown twice at 60 Hz
Display timing	720p60, 74.25 MHz pixel clock	Standard 720p timing
Core clock	~400 MHz	ASAP7 target
Compute units (CUs)	2	Identical, unified shaders
Lanes per CU	4	8 lanes total
Pixel work unit	One 2×2 quad per CU wave	4 pixels = 4 lanes
Registers per CU	16 scalar + 16 vector	s0 always reads 0
Instruction size	32 bits, fixed	Custom ISA (section 6)
Math	Fixed-point (section 4)	No floating-point units
Color buffer	RGBA8, 32 bits per pixel	Double buffered
Depth / stencil	16-bit depth + 8-bit stencil	Separate buffers
Textures	Up to 1024 × 1024, power-of-two, mipmapped	Bilinear filtering
External memory	DRAM, 16 MB minimum (32 MB recommended)	Layout in section 9

3. Top-level architecture

Work enters as a command list in DRAM, flows down a 7-stage pipeline, and leaves as pixels written back to DRAM, which the display engine streams to the screen. Vertex and pixel shading are the same 2 compute units running different programs (unified shaders).



Top-level architecture

The pipeline runs in this order: command processor, input assembler, vertex shading, primitive unit, rasterizer, pixel shading, render output unit. The memory system and the display path sit beside it. Every block's details are in section 8.

4. Number formats

All pipeline math uses fixed-point integers; "16.16" means 16 integer bits and 16 fraction bits in one 32-bit word, so 1.0 is stored as 65,536. These formats match the golden reference model (`fixed_point.py`) bit for bit.

Value	Format	Bits	Range / note
Shader registers, vertex positions, (u, v)	16.16 signed	32	±32,768, step 1/65,536
Screen positions	12.4 signed	16	4 subpixel bits (1/16 pixel)
Edge-function values	integer	40	Exact, no rounding
1 / triangle area (setup)	0.48 scaled	48	One divide per triangle
Barycentric weights	0.16	16	0 to 1 inside the triangle
Closeness (1/z)	0.16	16	Requires z > 1.0 (near plane)
Depth buffer	unsigned 16-bit	16	Stores closeness; larger = nearer
Stencil	unsigned 8-bit	8	
Color channel	unsigned 8-bit	8	Packed RGBA8 in 32 bits
Blend / filter fraction	unsigned 0.8	8	Bilinear and alpha blending
Angle (sin/cos table)	10-bit index	10	1,024 steps per circle

Fixed-point rules

- Multiply: `(a × b) >> 16`, using a 64-bit internal product.
- Right shifts are arithmetic (sign-preserving) and round toward minus infinity.
- Reciprocal: `2^32 / a`; the hardware reciprocal unit must reproduce the reference result exactly (algorithm defined in the block spec).
- 8-bit blend: `(c1 × (256 − f) + c2 × f) >> 8`.

5. Programming model

Shaders run as **waves** of 4 threads, one thread per lane, all executing the same instruction in lockstep. A vertex wave processes up to 4 vertices; a pixel wave processes one 2×2 pixel quad. In v0.1 each CU runs one wave at a time; several waves

per CU (to hide DRAM delay) is a later upgrade.

Registers (per wave)

Register	Count	Width	Note
Scalar s0–s15	16	32 bits	One value shared by the wave; s0 always reads 0
Vector v0–v15	16	4 × 32 bits	One value per lane
Program counter	1	32 bits	Byte address, 4-byte aligned
Execution mask	1	4 bits	1 = lane active; inactive lanes write nothing

Execution mask. For pixel waves the rasterizer sets the mask from triangle coverage. Uncovered lanes are **helper lanes**: the rasterizer still fills their inputs (v0–v10), so texture coordinates taken straight from the inputs are valid across the quad. But lanes outside the mask do not compute: their registers keep old values. **Rule for v0.1 shaders:** call `V_TEX` on computed coordinates before `V_KILL`, or the quad's mip level is picked from stale values (found by the textured-quad test). `V_KILL` clears mask bits.

Lane order in a quad: lane 0 = top-left, lane 1 = top-right, lane 2 = bottom-left, lane 3 = bottom-right.

Values at program start

Register	Vertex wave	Pixel wave
s1	Constant buffer address	Constant buffer address
s2	Index of first vertex in the batch	Primitive (triangle) number
v0 – v7	Vertex attributes ATTR_IN0–7 (16.16)	—
v0	—	Pixel x (integer)
v1	—	Pixel y (integer)
v2	—	Closeness 1/z (0.16)
v3 – v10	—	ATTR0–7, perspective-correct (16.16)

Shader outputs (via `EXPORT`)

Shader	Export target	Meaning
Vertex	POS_X, POS_Y, POS_Z	Camera-space position (16.16); projection is done by the primitive unit
Vertex	ATTR0 – ATTR7	Values to interpolate (colors, u, v, lighting)
Pixel	COLOR	Final color, packed RGBA8

A shader ends with `HALT`. Special values (CU number, wave type, frame number) are read with `S_SPR`.

6. Instruction set (ISA v0.1)

The ISA has 52 instructions in 4 encoding formats, all 32 bits wide with the opcode in bits 31–26. Scalar instructions start with `S_`, vector instructions with `V_`; vector instructions act on all active lanes at once.

Encoding formats

Format	31–26	25–22	21–18	17–14	13	12–9	8–0
R (register)	opcode	rd	ra	rb	BS	rc	0
I (immediate)	opcode	rd	ra	imm18 (17–0)			
B (branch)	opcode	—	ra	rb	off14 (13–0)		
J (jump)	opcode	off26 (25–0)					

- **BS = 1:** operand rb is read from the scalar file and broadcast to all lanes (for example, multiply every vertex by a matrix constant).
- **rc:** third source register, used by `V_MADFX`, `V_SEL`, `V_PACK`; texture slot for `V_TEX`.
- **imm18:** signed unless noted. **off14 / off26:** signed, counted in instructions, relative to the next instruction.

Control and special

Opcode	Mnemonic	Format	Operation
0x00	`NOP`	—	No operation

0x01	`HALT`	—	End of program; wave finishes
0x02	`S_JMP off`	J	PC = PC + 4 + off × 4
0x03	`S_BEQ sa, sb, off`	B	Branch if sa = sb
0x04	`S_BNE sa, sb, off`	B	Branch if sa ≠ sb
0x05	`S_BLT sa, sb, off`	B	Branch if sa < sb (signed)
0x06	`S_SPR sd, n`	I	sd = special value n: 0 CU number, 1 wave type (0 vertex, 1 pixel), 2 frame number, 3 lane count
0x07	`V_KILL va`	R	Lanes where va ≠ 0 leave the execution mask

Scalar ALU and memory

Opcode	Mnemonic	Format	Operation
0x08	`S_ADD sd, sa, sb`	R	sd = sa + sb
0x09	`S_SUB sd, sa, sb`	R	sd = sa - sb
0x0A	`S_AND sd, sa, sb`	R	Bitwise AND
0x0B	`S_OR sd, sa, sb`	R	Bitwise OR
0x0C	`S_XOR sd, sa, sb`	R	Bitwise XOR
0x0D	`S_SHL sd, sa, sb`	R	sd = sa << (sb & 31)
0x0E	`S_SHR sd, sa, sb`	R	Logical shift right
0x0F	`S_SRA sd, sa, sb`	R	Arithmetic shift right
0x10	`S_MUL sd, sa, sb`	R	Low 32 bits of sa × sb
0x11	`S_MULFX sd, sa, sb`	R	(sa × sb) >> 16
0x12	`S_SLT sd, sa, sb`	R	sd = 1 if sa < sb (signed), else 0
0x13	`S_ADDI sd, sa, imm`	I	sd = sa + imm18
0x14	`S_ORI sd, sa, imm`	I	sd = sa OR imm16 (zero-extended); sets a 16.16 fraction
0x15	`S_LUI sd, imm`	I	sd = imm16 << 16; sets a 16.16 integer part
0x18	`S_LOAD sd, [sa + imm]`	I	Load 32-bit word from memory
0x19	`S_STORE ss, [sa + imm]`	I	Store 32-bit word (ss in the rd field)

Vector ALU (all R format with BS allowed, unless noted)

Opcode	Mnemonic	Operation (per active lane)
0x20	`V_ADD vd, va, vb`	vd = va + vb
0x21	`V_SUB vd, va, vb`	vd = va - vb
0x22	`V_MUL vd, va, vb`	Low 32 bits of va × vb
0x23	`V_MULFX vd, va, vb`	(va × vb) >> 16
0x24	`V_MADFX vd, va, vb, vc`	vd = vc + ((va × vb) >> 16); one row of a matrix transform
0x25	`V_MIN vd, va, vb`	Signed minimum
0x26	`V_MAX vd, va, vb`	Signed maximum
0x27	`V_AND vd, va, vb`	Bitwise AND
0x28	`V_OR vd, va, vb`	Bitwise OR
0x29	`V_XOR vd, va, vb`	Bitwise XOR
0x2A	`V_SHL vd, va, vb`	Shift left by (vb & 31)
0x2B	`V_SHR vd, va, vb`	Logical shift right
0x2C	`V_SRA vd, va, vb`	Arithmetic shift right
0x2D	`V_SLT vd, va, vb`	vd = 1 if va < vb (signed), else 0

0x2E	`V_SEL vd, va, vb, vc`	vd = va if vc ≠ 0, else vb (no branching)
0x2F	`V_ADDI vd, va, imm`	vd = va + imm18 (I format)
0x30	`V_BCAST vd, sa`	Every lane = scalar sa (I format, ra = scalar)
0x31	`V_LANEID vd`	vd = lane number 0–3

Special function unit and vector memory

Opcode	Mnemonic	Format	Operation (per active lane)
0x34	`V_RCP vd, va`	R	vd = 1 / va (16.16), bit-exact with the reference
0x35	`V_RSQ vd, va`	R	vd = 1 / √va (16.16), for normalizing lighting vectors
0x38	`V_LOAD vd, [va + imm]`	I	Each lane loads a 32-bit word from its own address
0x39	`V_STORE vs, [va + imm]`	I	Each lane stores to its own address (vs in rd field)

Graphics

Opcode	Mnemonic	Format	Operation
0x3A	`V_TEX vd, va, vb, slot`	R	Sample texture slot (rc) at u = va, v = vb (16.16); bilinear + mipmap; result RGBA8. Pixel waves pick the mip level from the quad; vertex waves use level 0
0x3B	`V_CMUL vd, va, vb`	R	Per-channel multiply of two RGBA8 values: (a × (b + 1)) >> 8, so multiplying by white (255) leaves a color unchanged
0x3C	`V_CADD vd, va, vb`	R	Per-channel add of two RGBA8 values, clamped at 255
0x3D	`V_PACK vd, va, vb, vc`	R	RGBA8 from r = va, g = vb, b = vc (each clamped 0–255), alpha = 255
0x3E	`V_UNPK vd, va, ch`	I	vd = channel ch of RGBA8 va (0 R, 1 G, 2 B, 3 A)
0x3F	`EXPORT target, va`	I	Send va to the pipeline. Targets: 0–2 POS_X/Y/Z, 8–15 ATTR0–7, 16 COLOR

Opcodes 0x16–0x17, 0x1A–0x1F, 0x32–0x33 and 0x36–0x37 are reserved.

Details fixed by the instruction simulator

- **Color layout in RGBA8:** R = bits 7–0, G = 15–8, B = 23–16, A = 31–24.
- `V_RCP`**:** 1/0 gives 0x7FFFFFFF (largest positive); a negative input gives a negative result.
- `V_RSQ`**:** result = integer square root of (2^48 / a); zero or negative input gives 0x7FFFFFFF.
- `V_TEX` **in a pixel wave:** all 4 lanes sample, helper lanes included (needed for the mip level); only active lanes are written.
- `V_LOAD` **/** `V_STORE`**:** inactive lanes do not access memory. All addresses must be 4-byte aligned.

Example: textured, lit pixel shader (7 instructions)

```
; inputs: v3 = u (ATTR0), v4 = v (ATTR1), v5 = brightness (ATTR2, 16.16)
V_TEX   v11, v3, v4, 0      ; texel RGBA8 from texture slot 0
S_ADDI  s3, s0, 8           ; s3 = 8 (shift amount)
V_SRA   v12, v5, s3         ; brightness 16.16 -> 0..256
V_PACK  v13, v12, v12, v12  ; gray RGBA8 (clamped at 255)
V_CMUL  v14, v11, v13       ; texel x brightness, per channel
EXPORT  COLOR, v14
HALT
```

7. Command list and state registers

The host writes a list of commands into DRAM and gives its address to the command processor, which reads and executes

them in order. Every command starts with a 32-bit header, followed by zero or more 32-bit payload words.

Command header: bits 31–24 = command code · bits 23–16 = number of payload words · bits 15–0 = parameter.

Code	Command	Parameter	Payload	Action
0x00	`NOP`	—	—	Nothing
0x01	`SET_STATE`	State register number	Value	Write one state register
0x02	`CLEAR`	Bit 0 color, bit 1 depth, bit 2 stencil	Color RGBA8, depth, stencil	Fill the selected buffers
0x03	`DRAW_TRIANGLES`	Bit 0 = indexed	Count, first index	Draw count/3 triangles
0x04	`SWAP_BUFFERS`	—	—	Wait for rendering to finish. The first SWAP turns the display on; every later SWAP waits until the display has flipped to the new frame at vertical blank (no tearing)
0x05	`FENCE`	—	Address, value	Write value to address once all earlier work is done
0x06	`WAIT_VBLANK`	—	—	Pause until the next display frame starts (no effect before the display is on)
0x07	`JUMP`	—	Address	Continue reading commands at address
0xFF	`END`	—	—	Stop and raise an interrupt to the host

State registers (written with `SET_STATE`)

Number	Name	Meaning	Default
0x00	VS_ADDR	Vertex program address	0
0x01	PS_ADDR	Pixel program address	0
0x02	CONST_ADDR	Constant buffer address (loaded into s1)	0
0x03	VB_ADDR	Vertex buffer address	0
0x04	VB_STRIDE	Bytes per vertex	32
0x05	VB_ATTRS	Input attributes per vertex (1–8, each 16.16)	3
0x06	IB_ADDR	Index buffer address (16-bit indices)	0
0x07	VS_OUT_ATTRS	Attributes the vertex shader exports (0–8)	0
0x10	FB_A_ADDR	Color buffer A	0x500000
0x11	FB_B_ADDR	Color buffer B	0x900000
0x12	DEPTH_ADDR	Depth buffer	0xD00000
0x13	STENCIL_ADDR	Stencil buffer	0xED0000
0x14	VIEWPORT_W	Width in pixels	1280
0x15	VIEWPORT_H	Height in pixels	720
0x16	PROJ_FOCAL	Projection focal length (16.16)	—
0x17	PROJ_CX	Screen center x (16.16)	640.0
0x18	PROJ_CY	Screen center y (16.16)	360.0
0x19	NEAR_Z	Near plane distance (16.16, at least 1.0)	1.0
0x20	CULL_MODE	0 none, 1 back faces, 2 front faces	1
0x21	DEPTH_MODE	0 off, 1 test and write, 2 test only	1
0x22	STENCIL_CTRL	Bits 1–0 function (always, equal,	0

		not equal); bits 3–2 pass operation (keep, replace, increment); bits 15–8 reference; bits 23–16 mask	
0x23	BLEND_MODE	0 off, 1 alpha, 2 additive	0
0x24	COLOR_MASK	Bits 3–0 = write R, G, B, A	0xF
0x30–0x33	TEX0–3_ADDR	Texture slot base address (level 0; smaller levels follow in order)	0
0x34–0x37	TEX0–3_INFO	Bits 3–0 size as log2 (3–10); bits 7–4 mip levels; bit 8 format (0 RGBA8, 1 RGB565); bit 9 wrap (0 repeat, 1 clamp); bit 10 filter (0 point, 1 bilinear); bit 11 mipmaps on	0

Example: one frame

```
SET_STATE  VS_ADDR, 0x010000      ; vertex program
SET_STATE  PS_ADDR, 0x010400      ; pixel program
SET_STATE  TEX0_ADDR, 0x100000    ; cube texture
SET_STATE  TEX0_INFO, 0x0C87      ; 128x128, 8 levels, RGBA8, repeat, bilinear, mipmaps
CLEAR      color+depth, 0xFF1E1E1E, 0, 0
DRAW_TRIANGLES  indexed, 36, 0    ; one cube = 12 triangles
SWAP_BUFFERS
END
```

8. Block specifications

Each block below lists its job, its inputs and outputs, and its v0.1 sizes. Throughput figures are design targets, checked in section 10.

8.1 Host interface

- **Job:** lets the host start work and read status.
- **Registers:** CMD_ADDR (command list address), CMD_START (write 1 to start), STATUS (busy, idle, error), IRQ_STATUS / IRQ_CLEAR, FRAME_COUNT.
- **Bus:** 32-bit register bus; AXI4-Lite on the FPGA.

8.2 Command processor

- **Job:** reads the command list, keeps the state registers, starts draws, handles `SWAP_BUFFERS`, `FENCE`, `END`.
- **Inputs:** command words from L2 in 64-byte bursts. **Outputs:** state to all blocks, draw requests to the input assembler, interrupt to the host.
- **Sizes:** 16-word command FIFO; state registers per section 7.

8.3 Input assembler

- **Job:** reads 16-bit indices and vertex data, and groups vertices into batches of 4 (one vertex wave).
- **Outputs:** vertex batches to the work distributor; the triangle list (which 3 exported vertices form each triangle) to the primitive unit.
- **v0.1 rule:** every index is shaded, with no reuse of already-shaded vertices (up to 3× extra vertex work). A vertex reuse cache is a later upgrade.

8.4 Work distributor

- **Job:** gives vertex batches and pixel quads to whichever CU is free.
- **Policy:** pixel quads first (to drain the pipeline); vertex batches when no quad is waiting. Each quad gets a sequence number so the render output unit can keep triangle order.
- **Sizes:** 4 KB shared instruction cache with 2 read ports (one per CU).

8.5 Compute unit (× 2)

- **Pipeline:** 5 stages: fetch, decode, register read, execute, write back. v0.1 stalls on register hazards instead of forwarding.
- **Register files:** scalar 16 × 32 bits (2 read, 1 write); vector 16 × 4 × 32 bits (3 read, 1 write, for `V_MADFX` and `V_SEL`).

- **Lanes:** 4, each with a 32-bit ALU and a 32 × 32 multiplier (64-bit product for `MULFX`).
- **Special function unit:** one per CU, shared by the 4 lanes; reciprocal and reciprocal square root, 1 lane per cycle.
- **Load/store unit:** 1 lane per cycle in v0.1, through L2.
- **Target:** 1 instruction per cycle per CU when not stalled.

8.6 Texture unit and texture cache (one per CU)

- **Job:** executes `V_TEX`: texel address, wrap or clamp, bilinear blend of 4 texels, RGB565 to RGBA8 conversion (alpha 255).
- **Mip level:** pixel waves compare (u, v) of lane 0 with lane 1 (one pixel right) and lane 2 (one pixel down), one level per quad; footprint = largest absolute difference × texture size; level = position of its highest 1 bit. Vertex waves use level 0.
- **Cache:** 4 KB, 64-byte lines.
- **Target:** 1 bilinear sample per cycle, so one quad in 4 cycles.

8.7 Primitive unit

- **Job:** assembles each triangle from 3 exported vertices, culls, projects to the screen.
- **Near plane:** v0.1 discards any triangle with a vertex closer than NEAR_Z. Full clipping is a later upgrade.
- **Projection:** 1/z per vertex from its reciprocal unit; screen x = PROJ_CX + x × FOCAL / z, screen y = PROJ_CY - y × FOCAL / z; result in 12.4.
- **Culling:** sign of the triangle's edge-function area against CULL_MODE; zero-area triangles are discarded.
- **Outputs:** per vertex: screen x, y (12.4), closeness (0.16), and each attribute × closeness (for perspective correction).

8.8 Rasterizer

- **Setup:** edge equations, bounding box clamped to the viewport, one 1/area per triangle (48-bit precision).
- **Traversal:** walks the bounding box in 2×2 quad steps, row by row; coverage from exact integer edge tests at pixel centers. Shared edges follow the **top-left rule**, so no pixel is drawn twice or missed. A pixel center exactly on an edge (edge value 0) counts as inside only for a left edge (inside lies to its right) or a top edge (horizontal, inside below it); for other edges the value must be greater than 0. In hardware this is a per-edge bias of 0 or -1 chosen once during setup.
- **Early depth test:** checks closeness against the depth buffer before shading; fully hidden quads are dropped.
- **Interpolation:** weights (0.16), closeness, then each attribute = (blended attribute × closeness) × 1/closeness, using one reciprocal per pixel.
- **Target:** 1 quad every 4 cycles (limited by 1 reciprocal per cycle).

8.9 Render output unit

- **Job:** final per-pixel tests and the write to memory.
- **Order:** an 8-entry reorder buffer puts quads back in sequence-number order before writing.
- **Tests:** depth (new closeness > stored, per DEPTH_MODE), stencil (per STENCIL_CTRL).
- **Blending:** alpha: out = (src × a′ + dst × (256 - a′)) >> 8 per channel, where a′ = a + (a >> 7) maps alpha 0–255 onto 0–256, so alpha 255 gives exactly src and alpha 0 exactly dst; additive: per-channel add clamped at 255; color mask.
- **ROP cache:** 4 tiles of 8 × 8 pixels (color, depth, stencil), about 1.8 KB.

8.10 L2 cache and memory controller

- **L2 cache:** 64 KB shared, 64-byte lines.
- **Priority:** display engine > render output unit > rasterizer > CU sides (a CU or its texture unit, round-robin) > input assembler > command processor.
- **DRAM:** 16-bit DDR through a PHY (licensed IP, or the FPGA vendor's controller). Simulation uses a DRAM model with fixed latency.
- **Target:** at least 2 GB/s sustained.

8.11 Display engine

- **Timing:** 720p60: 1280 × 720 active, 1650 × 750 total, 74.25 MHz pixel clock.
- **Job:** reads the front buffer in 64-byte bursts into a 2-line FIFO (about 10 KB); swaps front and back buffers at vertical blank.
- **Outputs:** 24-bit RGB with HSYNC, VSYNC and data-enable; in simulation, each frame is saved as a PNG.

8.12 Upscaler (reserved)

- **Job (future):** 720p input to 1080p output (1920 × 1080 at 60 Hz, 148.5 MHz pixel clock), a 1.5× scale.
- **Plan:** bypassed in v1; then a bilinear filter with line buffers; then a small AI model.

9. Memory map

Everything fits in 16 MB of DRAM (0x000000–0xFFFFFF) with about 1.4 MB spare. Addresses are byte addresses, words are little-endian, and every region starts on a 64-byte boundary.

Start	End	Size	Contents
0x000000	0x00FFFF	64 KB	Command lists
0x010000	0x01FFFF	64 KB	Shader programs (16,384 instructions)
0x020000	0x02FFFF	64 KB	Constant buffers (matrices, light values)
0x030000	0x0FFFFF	832 KB	Vertex and index buffers
0x100000	0x4FFFFF	4 MB	Textures with their mipmaps
0x500000	0x883FFF	3.52 MB	Color buffer A (1280 × 720 × 4 bytes)
0x884000	0x8FFFFF	496 KB	Free
0x900000	0xC83FFF	3.52 MB	Color buffer B
0xC84000	0xCFFFFF	496 KB	Free
0xD00000	0xEC1FFF	1.76 MB	Depth buffer (16 bits per pixel)
0xEC2000	0xECFFFF	56 KB	Free
0xED0000	0xFB0FFF	900 KB	Stencil buffer (8 bits per pixel)
0xFB1000	0xFFFFFF	316 KB	Free

- **Pixel address:** base + (y × 1280 + x) × bytes per pixel.
- **Texture capacity:** the 4 MB region holds about 11 textures of 256 × 256 RGBA8 with mipmaps, or one 1024 × 1024 RGB565 texture with mipmaps (about 2.7 MB).
- **Mipmap layout:** level 0 at the slot address, each smaller level directly after the previous one; the driver builds each level as the rounded average of 2×2 texels, (a + b + c + d + 2) >> 2 per channel.
- A 32 MB DRAM doubles the texture space without changing any other address.

10. Performance budget

At 400 MHz every compute and pixel resource runs at 20% or less of capacity, and DRAM traffic needs about 1.05 GB/s of the 2 GB/s target. The real risk is waiting for DRAM, not raw speed.

Assumptions: 1280 × 720 at 30 fps; each pixel drawn twice on average (overdraw 2); 10 shader instructions and 1 texture sample per pixel; 10,000 triangles per frame, 40 instructions per vertex, no vertex reuse.

Resource	Needed	Capacity at 400 MHz	Used
Pixel shading	553 M lane-instructions/s	3,200 M (8 lanes)	17%
Vertex shading	36 M lane-instructions/s	shared with the line above	~1%
Rasterizer	13.8 M quads/s	100 M quads/s	14%
Texture sampling	55.3 M samples/s	800 M/s (2 units)	7%

DRAM traffic	Bandwidth
Display scan-out (720p60, 32-bit)	221 MB/s
Color writes	221 MB/s
Depth reads + writes (16-bit)	221 MB/s
Texture misses (assumed 4 bytes per pixel after the cache)	221 MB/s
Clearing color + depth each frame	166 MB/s
Total	**~1.05 GB/s**

Main risk: with one wave per CU, every DRAM access stalls the whole CU. If simulation shows stalls eating the headroom, the first fix is several waves per CU (section 12).

11. Verification plan

The hardware passes only when its output matches the fixed-point Python reference model bit for bit: every color pixel, depth value and stencil value. A float model is never used for pass/fail, because its rounding differs at edges.

Reference models

- **Golden pipeline model:** `fixed_point.py` grown into a full model that reads a command list and writes frames, depth and stencil.
- **Instruction simulator:** Python model of the ISA; the compute unit RTL must match it instruction by instruction (registers and memory after every step).
- **Required update:** the current reference counts pixels exactly on an edge as inside; it must adopt the top-left rule (section 8.8) before RTL comparison.

Test levels

1. **Instruction tests:** every opcode, including edge cases (largest and negative values, shifts of 0 and 31, reciprocal of tiny values, saturating color math).
2. **Block tests:** rasterizer coverage, texture sampler, render output unit, each driven alone and checked against the matching part of the model.
3. **Scene tests:** whole GPU at 64 × 36 on every change; at 1280 × 720 before each milestone.

Scene tests

Scene	What it checks
Single flat triangle	Edge functions, coverage
Color-blended triangle	Weights and interpolation
Two triangles sharing an edge	Top-left rule: no gaps, no double-drawn pixels
Spinning cube	Vertex shading, projection, back-face culling
Two overlapping cubes	Depth test and depth buffer
Textured cube	Perspective-correct texturing
Long textured floor	Bilinear filtering and mip level selection
Transparent quad over cube	Alpha blending, triangle order
Stencil window	Stencil test and operations
Triangle crossing the near plane	Near-plane discard
Two frames with SWAP_BUFFERS and FENCE	Double buffering, command handling

Tools: Python for the models; Icarus Verilog or Verilator for fast RTL simulation; Cadence Xcelium where available; results compared automatically by script.

12. Roadmap and open questions

The build runs in 11 phases, each ending in something that works and can be shown; the Python reference (Step 1) is already complete.

1. **Reference and tools:** full golden model (command list in, frames out) with the top-left rule; ISA instruction simulator; assembler. *Proof:* all scene tests render in Python.
2. **One compute unit in RTL,** non-pipelined first. *Proof:* matches the instruction simulator.
3. **Second CU and work distributor.** *Proof:* a triangle drawn by a shader program, from RTL.
4. **Pipelined compute units.** *Proof:* same results, fewer cycles.
5. **Primitive unit, rasterizer, render output unit with depth.** *Proof:* two overlapping cubes, bit-exact.
6. **Texture unit, texture cache, reciprocal unit.** *Proof:* textured cube and mipmapped floor, bit-exact.
7. **Blending and stencil.** *Proof:* transparency and stencil scenes.
8. **Command processor, input assembler, L2 and memory controller with a DRAM model.** *Proof:* full 1280 × 720 frames from a command list.
9. **Display engine.** *Proof:* correct 720p60 timing in simulation.
10. **ASAP7 physical design** (Genus, Innovus, Tempus). *Proof:* timing closed near 400 MHz, with area and power numbers.
11. **FPGA demonstration.** *Proof:* live 3D on a monitor over HDMI.

Later upgrades: several waves per CU, full near-plane clipping, vertex reuse cache, upscaler (bilinear, then AI).

Open questions

- [ ] One wave per CU: enough, or are several waves needed from the start? Decide after Phase 8 stall measurements.

- [ ] Near plane: is discarding whole triangles acceptable for the test scenes, or is clipping needed earlier?
- [ ] 16-bit depth: enough precision for long scenes, or move to 24-bit?
- [ ] Stencil: keep a separate 8-bit buffer, or pack depth and stencil into one 32-bit word?
- [ ] Fixed-point only, or add floating-point later for vertex math?
- [ ] FPGA board choice: needs DRAM and HDMI output.
- [ ] Upscaler: which bilinear design first, and which AI model size later?
- [ ] Helper lanes: should lanes outside the mask keep computing, with the mask blocking only stores and `EXPORT` (as real GPUs do)? That removes the v0.1 "V_TEX before V_KILL" rule. Decide before the hardware rasterizer (Phase 5).

13. Implementation status (v0.2)

The complete GPU is implemented in RTL and runs a command list on its own, from command processing to video output. All 12 test suites pass: every block matches the Python golden model bit for bit, and the complete GPU matches it on every pixel of every frame and on the video output (correct pictures in order, no tearing, no display underflow).

Block	RTL file	Verified by
Compute unit (all 52 instructions)	`cu.v`	30 programs, every register, mask, memory word and export
Special function unit	inside `cu.v`	Algorithms proven on 120,014 values, then 30-program test
Texture unit	`tex_unit.v`	416 requests (16 directed, 400 random) with random memory stalls
Rasterizer	`rasterizer.v`	150 random triangles, early depth off and on, about 21,000 quads
Primitive unit	`prim_unit.v`	720 random triangles in 12 draws with random settings
Render output unit	`rop.v`	2,500 random clears and quads; whole color, depth and stencil buffers
Display engine	`display_engine.v`	Cycle-exact on a small screen with flips; real 720p60 timing; underflow alarm
Complete GPU	`gpu_top.v`	2 scenes, 4 frames, from command lists; frames and video output

Decisions made during implementation

- Closeness (1/z) saturates at 32 bits; projection results and attribute × 1/z wrap to 32 bits, exactly as the hardware registers do.
- A triangle enters the rasterizer only after every pixel of the previous triangle is written, so early depth always sees finished depth values.
- Vertex shader results are kept in a 256-vertex vertex output buffer per draw; larger draws raise an error.
- The render output unit uses the export mask at `EXPORT COLOR` time.
- The first `SWAP_BUFFERS` turns the display on; later ones wait for the flip.
- Signal names avoid SystemVerilog keywords (the name `cover` broke the first rasterizer compile).

Measured performance (160 × 90 test scenes, simulation cycles for 2 frames, display included)

Scene	Cycles	Main cost
UV cubes	467,943	Rasterizer reciprocals
Textured cube over floor	1,138,546	Rasterizer reciprocals (about 140 of 160 cycles per quad)

Not yet implemented

- Stencil test and stencil operations (stencil buffer clear only).
- Pipelined compute units; several waves per CU; texture cache and L2 cache.
- DRAM controller and PHY (the testbench provides a single-cycle memory).
- Separate display clock with an asynchronous FIFO.
- Full near-plane clipping; vertex reuse cache; upscaler.
- Physical design on ASAP7 and the FPGA demonstration.